We were told by a TA that we don’t need to provide a write-up for the experiments and just do the take-home exercise.

Chris Jiang 400322193
Sohel Saini 400327615

Exercise:

We began by using the code from Experiment 5 as the foundation for completing the take-home exercise. We increased the bit count of the data register to accommodate a flag (mode) that determines whether the character should be in uppercase or lowercase. To implement this, we replaced the 0 in PS2_to_LCD_ROM with the mode variable, which is initially set to lowercase mode. The core implementation occurs in the S_IDLE state. When the PS2_code corresponds to either the left or right shift key, the flag is toggled to uppercase mode (since it starts in lowercase by default). The rest of the operations are handled using conditional checks to ensure the data counter functions correctly. We made further adjustments in the section of the code where the PS2_code is loaded into the shift register. Specifically, we added the mode flag before PS2_code to correctly reference the memory. To handle the "0" and "9" cases, we created logic to manage the top and bottom lines, as well as logic for the ones and tens digits. In the Resetn state, we set the ones/tens digits to F (4'hF) to display a default null position. Next, in the S_IDLE state, we created if statements based on whether we were working with the top or bottom line. If we were on the top line and the data_counter was less than 15, we added the mode and PS2_code to the top-line logic. To handle the "9", we checked if its make code was present. We separated the ones digit by taking the data_counter, performing a modulus operation, and adding 1 to ensure the count remained within the 16-bit limit. Similarly, for the tens digit, we divided the data_counter by 10. After incrementing the data_counter, if it exceeded 15, we reset it and transitioned the state to S_LCD_WAIT_ROM_UPDATE. To make the sequence go backwards we looped the counter to subtract it by 1 each time. With this an error showing where 00 would not get displayed instead it would be 01 twice, we fixed this by adding a line of code where ones are subtracted by 1 (1’d1). An implemented flag was put in place to catch the msb for 9 and cut off any other 9 inputted afterwards. We applied the same logic for handling the "0", adjusting it to account for the bottom line, adding one to the ones counter to incorporate the last number (15), and took away the flag from the msb in order to catch the lsb for 0.